

Timestamp Key Encoding + Refactor

In this chapter, you will:

- Refactor your implementation to use key+ts representation.
- Make your code compile with the new key representation.

To run test cases,

```
cargo x copy-test --week 3 --day 1
cargo x scheck
```

Note: The MVCC subsystem is not fully implemented until week 3 day 2. You only need to pass week 3 day 1 tests and all week 1 tests at the end of this day. Week 2 tests won't work because of compaction.

Task 0: Use MVCC Key Encoding

You will need to replace the key encoding module to the MVCC one. We have removed some interfaces from the original key module and implemented new comparators for the keys. If you followed the instructions in the previous chapters and did not use `into_inner` on the key, you should pass all test cases on day 3 after all the refactors. Otherwise, you will need to look carefully on the places where you only compare the keys without looking at the timestamps.

Specifically, the key type definition has been changed from:

```
pub struct Key<T: AsRef<[u8]>>(T);
```

...to:

```
pub struct Key<T: AsRef<[u8]>>(T /* user key */, u64 /* timestamp */);
```

...where we have a timestamp associated with the keys. We only use this key representation internally in the system. On the user interface side, we do not ask users to provide a timestamp, and therefore some structures still use `&[u8]` instead of `KeySlice` in the engine. We will cover the places where we need to change the signature of the functions later. For now, you only need to run,

```
cp mini-lsm-mvcc/src/key.rs mini-lsm-starter/src/
```

There are other ways of storing the timestamp. For example, we can still use the `pub struct Key<T: AsRef<[u8]>>(T);` representation, but assume the last 8 bytes of the key is the timestamp. You can also implement this as part of the bonus tasks.

Alternative key representation: | user_key (varlen) | ts (8 bytes) | in a single slice

Our key representation: | user_key slice | ts (u64) |

In the key+ts encoding, the key with a smallest user key and a largest timestamp will be ordered first. For example,

`("a", 233) < ("a", 0) < ("b", 233) < ("b", 0)`

Task 1: Encode Timestamps in Blocks

The first thing you will notice is that your code might not compile after replacing the key module. In this chapter, all you need to do is to make it compile. In this task, you will need to modify:

```
src/block.rs
src/block/builder.rs
src/block/iterator.rs
```

You will notice that `raw_ref()` and `len()` are removed from the key API. Instead, we have `key_ref` to retrieve the slice of the user key, and `key_len` to retrieve the length of the user key. You will need to refactor your block builder and decoding implementation to use the new APIs. Also, you will need to change your block encoding to encode the timestamps. In `BlockBuilder::add`, you should do that. The new block entry record will be like:

```
key_overlap_len (u16) | remaining_key_len (u16) | key (remaining_key_len) |
timestamp (u64)
```

You may use `raw_len` to estimate the space required by a key, and store the timestamp after the user key.

After you change the block encoding, you will need to change the decoding in both `block.rs` and `iterator.rs` accordingly.

Task 2: Encoding Timestamps in SSTs

Then, you can go ahead and modify the table format,

```
src/table.rs
src/table/builder.rs
src/table/iterator.rs
```

Specifically, you will need to change your block meta encoding to include the timestamps of the keys. All other code remains the same. As we use `KeySlice` in the signature of all functions (i.e., `seek`, `add`), the new key comparator should automatically order the keys by user key and timestamps.

In your table builder, you may directly use the `key_ref()` to build the bloom filter. This naturally creates a prefix bloom filter for your SSTs.

Task 3: LSM Iterators

As we use associated generic type to make most of our iterators work for different key types (i.e., `&[u8]` and `KeySlice<'_>`), we do not need to modify merge iterators and concat iterators if they are implemented correctly. The `LsmIterator` is the place where we strip the timestamp from the internal key representation and return the latest version of a key to the user. In this task, you will need to modify:

```
src/lsm_iterator.rs
```

For now, we do not modify the logic of `LsmIterator` to only keep the latest version of a key. We simply make it compile by appending a timestamp to the user key when passing the key to the inner iterator, and stripping the timestamp from a key when returning to the user. The behavior of your LSM iterator for now should be returning multiple versions of the same key to the user.

Task 4: Memtable

For now, we keep the logic of the memtable. We return a key slice to the user and flush SSTs with `TS_DEFAULT`. We will change the memtable to be MVCC in the next chapter. In this task, you will need to modify:

```
src/mem_table.rs
```

Task 5: Engine Read Path

In this task, you will need to modify,

```
src/lsm_storage.rs
```

Now that we have a timestamp in the key, and when creating the iterators, we will need to seek a key with a timestamp instead of only the user key. You can create a key slice with `TS_RANGE_BEGIN`, which is the largest ts.

When you check if a user key is in a table, you can simply compare the user key without comparing the timestamp.

At this point, you should build your implementation and pass all week 1 test cases. All keys stored in the system will use `TS_DEFAULT` (which is timestamp 0). We will make the engine fully multi-version and pass all test cases in the next two chapters.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.